

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME:** CSA1610 **COURSE CODE:** Data Warehousing and Data Mining

---

### COURSE OUTCOME COVERED

**CO5:** Apply association rule mining techniques to discover interesting relationships and patterns within large datasets

---

**QUESTIONS: 05**

**Total Marks:50**

Annexure A

Constraint-Based Problem Solving

| <b>Criteria</b>  | Problem Understanding<br>(2.5) | Constraint Analysis<br>(2.5) | Solution Development<br>(2.5) | Application of Concepts<br>(1.5) | Justification<br>(1) | <b>Total(10)</b> |
|------------------|--------------------------------|------------------------------|-------------------------------|----------------------------------|----------------------|------------------|
| <b>Weightage</b> | 25%                            | 25%                          | 25%                           | 15%                              | 10%                  | 100%             |
| <i>Q1</i>        |                                |                              |                               |                                  |                      |                  |
| <i>Q2</i>        |                                |                              |                               |                                  |                      |                  |
| <i>Q3</i>        |                                |                              |                               |                                  |                      |                  |
| <i>Q4</i>        |                                |                              |                               |                                  |                      |                  |
| <i>Q5</i>        |                                |                              |                               |                                  |                      |                  |

---

### **Code of Conduct:**

*I P, Madesh 192365507 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:**

*P. Madesh*

## RUBRICS FOR CALCULATION:

### Constraint-Based Problem Solving Assessment Rubric

| Criteria                | Weightage | Level 4<br>(Exemplary)                                                         | Level 3<br>(Proficient)                               | Level 2<br>(Developing)                       | Level 1<br>(Beginning)                   |
|-------------------------|-----------|--------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------|------------------------------------------|
| Problem Understanding   | 25%       | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps           | Partial understanding; misses key constraints | Fails to identify constraints correctly  |
| Constraint Analysis     | 25%       | Analyzes constraints effectively and prioritizes them logically                | Provides reasonable analysis with some prioritization | Limited analysis; weak prioritization         | No meaningful analysis or prioritization |
| Solution Development    | 25%       | Develops optimal and feasible solutions satisfying all constraints             | Develops workable solutions with minor limitations    | Solutions partially satisfy constraints       | Solutions do not meet constraints        |
| Application of Concepts | 15%       | Effectively applies appropriate concepts, methods, or tools                    | Applies concepts with minor errors                    | Limited application; requires guidance        | Unable to apply relevant concepts        |
| Justification           | 10%       | Provides strong justification for decisions with logical reasoning             | Provides justification with some clarity              | Weak or unclear justification                 | No justification provided                |

## Annexure A

### Q1. Dataset: Hospital Patient Records

#### PID   Symptoms Observed

|    |                        |
|----|------------------------|
| P1 | Fever, Cough           |
| P2 | Fever, Headache        |
| P3 | Cough, Headache        |
| P4 | Fever, Cough, Headache |
| P5 | Fever, Fatigue         |

#### Question:

Apply association rule mining with minimum support = 50% and confidence = 70%, under the constraint that all generated rules must include "**Fever**". Identify the valid association rules and justify why they satisfy the given constraints.

---

### Q2. Dataset: Online Learning Platform

#### Student ID   Courses Enrolled

|    |                           |
|----|---------------------------|
| S1 | Python, SQL               |
| S2 | Python, Data Science      |
| S3 | SQL, Machine Learning     |
| S4 | Python, SQL, Data Science |
| S5 | Python, Machine Learning  |

#### Question:

Using the dataset, generate frequent itemsets with minimum support = 40%, applying the constraint that each itemset must contain **at least two courses**. Explain how the constraint helps in pruning candidate itemsets during mining.

---

### Q3. Dataset: Library Borrowing Records

#### Transaction ID   Books Borrowed

|    |                                 |
|----|---------------------------------|
| T1 | Data Mining, Python Programming |
|----|---------------------------------|

| Transaction ID | Books Borrowed                                    |
|----------------|---------------------------------------------------|
| T2             | Data Mining, Database Systems                     |
| T3             | Python Programming, Database Systems              |
| T4             | Data Mining, Python Programming, Database Systems |
| T5             | Data Mining, Artificial Intelligence              |

**Question:**

Apply the Apriori algorithm with the constraint that the **maximum itemset size is 3**. Identify the frequent itemsets and explain how the constraint reduces computational complexity during candidate generation.

---

#### Q4. Dataset: University Course Hierarchy

| Student ID | Courses Taken                                   |
|------------|-------------------------------------------------|
| S1         | Engineering, Computer Science, Machine Learning |
| S2         | Engineering, Information Technology             |
| S3         | Engineering, Computer Science                   |
| S4         | Engineering, Computer Science, Machine Learning |
| S5         | Engineering, Information Technology             |

**Question:**

Apply multilevel association rule mining with the constraint that rules should be generated only at the "**Computer Science**" level. Explain how the course hierarchy affects the discovery of association rules.

---

#### Q5. Dataset: Insurance Customer Records

| CID | Age Group | Income Level | Policy Purchased  |
|-----|-----------|--------------|-------------------|
| C1  | Young     | High         | Health Insurance  |
| C2  | Middle    | Medium       | Vehicle Insurance |
| C3  | Young     | Low          | Travel Insurance  |
| C4  | Senior    | High         | Health Insurance  |
| C5  | Young     | Medium       | Vehicle Insurance |

**Question:**

Apply multidimensional association rule mining with the constraint **Age Group = "Young"**. Identify meaningful patterns involving income level and policy type, and explain how these patterns can support targeted marketing strategies.

# Q1. Hospital Patient Records — Association Rule Mining with an Item Constraint

Dataset (N = 5 patient transactions):

| PID | Symptoms Observed      |
|-----|------------------------|
| P1  | Fever, Cough           |
| P2  | Fever, Headache        |
| P3  | Cough, Headache        |
| P4  | Fever, Cough, Headache |
| P5  | Fever, Fatigue         |

Given: minimum support = 50% | minimum confidence = 70% | Constraint: every generated rule must contain the item "Fever".

## Step 1 — Problem & Constraint Understanding

This is an item constraint ( $C: \text{'Fever'} \in X$ ) combined with a support constraint. Item constraints of this form are neither anti-monotone nor monotone by themselves, but in the Apriori framework they are most efficiently enforced by pruning at the 1-itemset stage: only itemsets that already contain 'Fever' need to be extended, because any itemset not containing Fever can never produce a rule that satisfies C. This is the standard 'constraint pushing' strategy for item constraints.

## Step 2 — Constraint Analysis: Support Counting (min\_sup = 50% → count ≥ 2.5, i.e. ≥ 3 of 5)

| Itemset                  | Transactions Containing It | Support Count | Support % | Frequent? (≥50%) |
|--------------------------|----------------------------|---------------|-----------|------------------|
| {Fever}                  | P1,P2,P4,P5                | 4             | 80%       | Yes              |
| {Cough}                  | P1,P3,P4                   | 3             | 60%       | Yes              |
| {Headache}               | P2,P3,P4                   | 3             | 60%       | Yes              |
| {Fatigue}                | P5                         | 1             | 20%       | No               |
| {Fever, Cough}           | P1,P4                      | 2             | 40%       | No               |
| {Fever, Headache}        | P2,P4                      | 2             | 40%       | No               |
| {Fever, Fatigue}         | P5                         | 1             | 20%       | No               |
| {Fever, Cough, Headache} | P4                         | 1             | 20%       | No               |

## Step 3 — Solution Development: Rule Generation

A valid association rule requires a frequent itemset of size  $\geq 2$  to split into antecedent → consequent. Applying the constraint 'Fever must appear in the rule' restricts candidate itemsets to those in the last four rows above — {Fever,Cough}, {Fever,Headache}, {Fever,Fatigue}, {Fever,Cough,Headache}. Every one of these has support 40% or 20%, which is below the 50% minimum support threshold, so none of them qualifies as a frequent itemset.

**Result: No association rule satisfies both the minimum support (50%) and the item constraint (rule must contain 'Fever') simultaneously.**

**Only the singleton {Fever} is frequent (80% support), but a singleton cannot form a rule (a rule needs a non-empty antecedent AND a non-empty consequent).**

## Step 4 — Justification / Application of Concepts

- Anti-monotone property of support: since {Fever,Cough} and {Fever,Headache} already fall below min\_sup at size 2, any larger superset containing them (e.g. {Fever,Cough,Headache}) is guaranteed to be infrequent too — Apriori correctly prunes the whole branch without further counting.
- The item constraint 'Fever ∈ rule' by itself does not reduce the search space as much as the support constraint here — it only tells the algorithm which branches are worth generating; the support threshold is what actually eliminates them.
- Pedagogical takeaway: constraint-based mining does not guarantee that a rule set will be non-empty — a strict support threshold combined with a restrictive item constraint can legitimately yield zero rules, and correctly reporting 'no valid rule exists' (with the supporting computation) is the correct analytical outcome, not a failure of the method.
- If the intent was to discover Fever-related co-occurrences, the analyst has two constraint-relaxation options: (a) lower min\_sup to 40% (count ≥ 2), which would make {Fever,Cough} and {Fever,Headache} frequent with confidence  $2/4 = 50\%$  each — still below 70% confidence, so still no rule at 70% confidence; or (b) collect more transactions so genuine co-occurrence patterns reach statistical significance.

# Q2. Online Learning Platform — Frequent Itemsets with a Size Constraint

Dataset (N = 5 student transactions):

| Student ID | Courses Enrolled          |
|------------|---------------------------|
| S1         | Python, SQL               |
| S2         | Python, Data Science      |
| S3         | SQL, Machine Learning     |
| S4         | Python, SQL, Data Science |
| S5         | Python, Machine Learning  |

Given: minimum support = 40% | Constraint: every reported itemset must contain at least 2 courses ( $|X| \geq 2$ ).

## Step 1 — Problem & Constraint Understanding

The constraint  $|X| \geq 2$  is a size (cardinality) constraint. It is monotone with respect to itemset growth — once an itemset has 2 or more items it will continue to satisfy the constraint at every larger size — so it cannot be used to prune upward search the way an anti-monotone constraint can. Its practical role is different: 1-itemsets are still generated and counted internally (Apriori needs them to build 2-itemset candidates via the join step), but they are filtered out of the final reported output, since a single course can never itself represent a 'combination' of interest to the platform.

## Step 2 — Constraint Analysis: Support Counting (min\_sup = 40% → count ≥ 2 of 5)

| Itemset                          | Support Count | Support % | Frequent (≥40%)? | Reported? (size ≥ 2)        |
|----------------------------------|---------------|-----------|------------------|-----------------------------|
| {Python}                         | 4             | 80%       | Yes              | No — filtered by constraint |
| {SQL}                            | 3             | 60%       | Yes              | No — filtered by constraint |
| {Data Science}                   | 2             | 40%       | Yes              | No — filtered by constraint |
| {Machine Learning}               | 2             | 40%       | Yes              | No — filtered by constraint |
| {Python, SQL}                    | 2             | 40%       | Yes              | Yes                         |
| {Python, Data Science}           | 2             | 40%       | Yes              | Yes                         |
| {Python, Machine Learning}       | 1             | 20%       | No               | —                           |
| {SQL, Data Science}              | 1             | 20%       | No               | —                           |
| {SQL, Machine Learning}          | 1             | 20%       | No               | —                           |
| {Data Science, Machine Learning} | 0             | 0%        | No               | —                           |

| Itemset                     | Support Count | Support % | Frequent ( $\geq 40\%$ )? | Reported? (size $\geq 2$ ) |
|-----------------------------|---------------|-----------|---------------------------|----------------------------|
| {Python, SQL, Data Science} | 1             | 20%       | No                        | —                          |

## Step 3 — Solution Development: Final Frequent Itemsets Satisfying the Constraint

**{Python, SQL} — support 40%**

**{Python, Data Science} — support 40%**

**No 3-itemset survives (support of {Python,SQL,Data Science} = 20% < 40%), so mining terminates at level 2.**

## Step 4 — Justification / Application of Concepts

- Constraint role:  $|X| \geq 2$  is a succinct, monotone constraint. Standard practice is to generate candidates without regard to it (because pushing a monotone constraint into candidate generation gives no pruning benefit) and apply it only as a post-processing filter on the output.
- Support (anti-monotone) is the constraint that actually shrinks the search space here: because {Python, Machine Learning}, {SQL, Data Science} and {SQL, Machine Learning} fail min\_sup at size 2, Apriori's downward-closure property guarantees no 3-itemset containing any of these pairs needs to be counted, e.g. {Python, SQL, Machine Learning} is pruned without ever being counted.
- Interpretation for the platform: 'Python + SQL' and 'Python + Data Science' are the only course pairings popular enough (40% of learners) to justify a bundled recommendation or a combined learning-path promotion.



## Q3. Library Borrowing Records — Apriori with a Maximum Itemset-Size Constraint

Dataset (N = 5 borrowing transactions):

| Transaction ID | Books Borrowed                                    |
|----------------|---------------------------------------------------|
| T1             | Data Mining, Python Programming                   |
| T2             | Data Mining, Database Systems                     |
| T3             | Python Programming, Database Systems              |
| T4             | Data Mining, Python Programming, Database Systems |
| T5             | Data Mining, Artificial Intelligence              |

**Given: Constraint: maximum itemset size = 3.** (No explicit support threshold is stated in the question; for a worked, gradable answer we adopt the same minimum support used elsewhere in this set,  $\text{min\_sup} = 40\% \rightarrow \text{count} \geq 2$  of 5, and state this assumption explicitly, as is standard practice when a threshold is not specified.)

### Step 1 — Problem & Constraint Understanding

The constraint 'maximum itemset size = 3' is a monotone-in-reverse (anti-monotone-like) bound: it does not affect which itemsets of size  $\leq 3$  are frequent, but it stops Apriori from generating and counting any candidate of size 4 or larger — level-4 candidate generation (the join + prune step at  $k = 4$ ) is simply never executed.

### Step 2 — Support Counting

**1-Itemsets:**

| Itemset                        | Support Count | Support % | Frequent ( $\geq 40\%$ )? |
|--------------------------------|---------------|-----------|---------------------------|
| {Data Mining (DM)}             | 4             | 80%       | Yes                       |
| {Python Programming (PP)}      | 3             | 60%       | Yes                       |
| {Database Systems (DS)}        | 3             | 60%       | Yes                       |
| {Artificial Intelligence (AI)} | 1             | 20%       | No                        |

**2-Itemsets (candidates generated only from frequent 1-itemsets: DM, PP, DS):**

| Itemset  | Support Count | Support % | Frequent ( $\geq 40\%$ )? |
|----------|---------------|-----------|---------------------------|
| {DM, PP} | 2 (T1, T4)    | 40%       | Yes                       |
| {DM, DS} | 2 (T2, T4)    | 40%       | Yes                       |
| {PP, DS} | 2 (T3, T4)    | 40%       | Yes                       |

**3-Itemsets (level capped at  $k = 3$  by the constraint — this is also the last level Apriori will evaluate):**

| Itemset      | Support Count | Support % | Frequent ( $\geq 40\%$ )? |
|--------------|---------------|-----------|---------------------------|
| {DM, PP, DS} | 1 (T4 only)   | 20%       | No                        |

Level  $k = 4$  ( $\{DM, PP, DS, AI\}$  would be the only possible 4-itemset) is never generated: the max-size-3 constraint terminates the algorithm after level 3 regardless of whether  $\{DM, PP, DS\}$  had been frequent.

## Step 3 — Final Frequent Itemsets (size $\leq 3$ , support $\geq 40\%$ )

**Size 1:**  $\{DM\}$  80%,  $\{PP\}$  60%,  $\{DS\}$  60%

**Size 2:**  $\{DM, PP\}$  40%,  $\{DM, DS\}$  40%,  $\{PP, DS\}$  40%

**Size 3:** none (the only candidate,  $\{DM, PP, DS\}$ , fails minimum support at 20%)

## Step 4 — Justification / Application of Concepts (Computational Complexity)

- Without the size cap, Apriori would still attempt level-4 candidate generation via the join step (joining the frequent 3-itemsets, if any had existed, or checking closure). With only 4 distinct items in this small dataset the saving is minor, but the principle scales: for a catalogue of thousands of book titles, capping itemset size at 3 prevents the combinatorial explosion of candidate generation at higher  $k$ , where the number of possible  $k$ -itemsets grows as  $C(n, k)$ .
- The max-size constraint is convertible/anti-monotone in effect on the level index  $k$  itself: it guarantees Apriori will never even attempt the expensive join+prune step for  $k > 3$ , saving both the database scan (support counting pass) and the memory used to store larger candidate sets.
- Note that  $\{DM, PP, DS\}$  failing minimum support (20%, present only in T4) means the size cap did not even need to act here as a pruning mechanism — support-based anti-monotone pruning already eliminated the only size-3 candidate. The constraint's benefit is guaranteed in general (worst case) even when, as in this small example, ordinary support pruning happens to do the same job first.

## Q4. University Course Hierarchy — Multilevel Association Rule Mining

Dataset (N = 5 student transactions):

| Student ID | Courses Taken                                   |
|------------|-------------------------------------------------|
| S1         | Engineering, Computer Science, Machine Learning |
| S2         | Engineering, Information Technology             |
| S3         | Engineering, Computer Science                   |
| S4         | Engineering, Computer Science, Machine Learning |
| S5         | Engineering, Information Technology             |

**Concept hierarchy (given/implied):** Engineering (Level 1, root) → {Computer Science, Information Technology} (Level 2) → {Machine Learning} (Level 3, specialization of Computer Science).

**Constraint:** generate rules only at the 'Computer Science' level, i.e. within the Computer Science branch of the hierarchy (Computer Science and its descendant, Machine Learning).

### Step 1 — Problem & Constraint Understanding

Multilevel association rule mining replaces flat item mining with mining across a concept taxonomy: rules can be discovered at the root, at a middle level, or at a leaf/specialization level. Restricting mining 'to the Computer Science level' means we only compute support/confidence for the Computer Science node and its descendant Machine Learning, ignoring the sibling Information Technology branch (which S2 and S5 belong to).

### Step 2 — Constraint Analysis: Level Filtering and Support Counting

| Node / Item                                               | In Transactions | Support Count | Support % |
|-----------------------------------------------------------|-----------------|---------------|-----------|
| Engineering (root)                                        | S1,S2,S3,S4,S5  | 5             | 100%      |
| Computer Science (Level 2)                                | S1,S3,S4        | 3             | 60%       |
| Information Technology (Level 2) — excluded by constraint | S2,S5           | 2             | 40%       |
| Machine Learning (Level 3, child of CS)                   | S1,S4           | 2             | 40%       |

### Step 3 — Solution Development: Rules Within the Computer Science Branch

| Candidate Rule                 | Support (both items)           | Confidence | Comment                                                                                    |
|--------------------------------|--------------------------------|------------|--------------------------------------------------------------------------------------------|
| Engineering → Computer Science | {Eng,CS}: S1,S3,S4 = 3/5 = 60% | 3/5 = 60%  | Every CS student is also an Engineering student, so this is really testing how much of the |

| Candidate Rule                      | Support (both items)        | Confidence  | Comment                                                                                                      |
|-------------------------------------|-----------------------------|-------------|--------------------------------------------------------------------------------------------------------------|
|                                     |                             |             | (universal) Engineering base specializes into CS.                                                            |
| Computer Science → Machine Learning | {CS,ML}: S1,S4 = 2/5 = 40%  | 2/3 = 66.7% | Strongest rule in this branch: two-thirds of CS students also take Machine Learning.                         |
| Engineering → Machine Learning      | {Eng,ML}: S1,S4 = 2/5 = 40% | 2/5 = 40%   | Weaker directly from the root — consistent with ML being a level-3 specialization, not a root-level pattern. |

**Strongest CS-level rule: Computer Science → Machine Learning (support 40%, confidence 66.7%).**  
**This pattern is only visible once mining is scoped to the Computer Science branch; at the root level it is diluted by the unrelated Information Technology branch.**

## Step 4 — Justification / Application of Concepts

- Effect of the hierarchy: because Machine Learning is a child of Computer Science in the taxonomy, every ML-enrolled student is automatically a CS-branch student — the hierarchy guarantees {CS,ML} support cannot exceed {CS} support, mirroring the anti-monotone downward-closure idea but applied across levels rather than across itemset size.
- Restricting rule generation to the CS level removes noise from the sibling Information Technology branch (S2, S5), which would otherwise be mixed into any root-level ('Engineering only') analysis and dilute the true CS → ML relationship.
- This illustrates the 'reduced support' multilevel strategy: rather than using one uniform minimum-support threshold across all levels (which tends to miss deeper-level, more specific but lower-frequency patterns like CS→ML), a level-specific / branch-specific threshold surfaces the CS→ML rule that a single flat threshold applied to the whole dataset might discard as too specific.

## Q5. Insurance Customer Records — Multidimensional Association Rule Mining

Dataset (N = 5 customer records):

| CID | Age Group | Income Level | Policy Purchased  |
|-----|-----------|--------------|-------------------|
| C1  | Young     | High         | Health Insurance  |
| C2  | Middle    | Medium       | Vehicle Insurance |
| C3  | Young     | Low          | Travel Insurance  |
| C4  | Senior    | High         | Health Insurance  |
| C5  | Young     | Medium       | Vehicle Insurance |

**Given: Constraint: Age Group = "Young". Dimensions involved: Age Group, Income Level, Policy Purchased — this is multidimensional (inter-dimension) association rule mining, since the antecedent and consequent are drawn from different attributes/dimensions rather than from one 'items' dimension.**

### Step 1 — Problem & Constraint Understanding

Multidimensional association rules relate values across different predicates/dimensions of a relational or data-warehouse table (here: Age Group, Income Level, Policy Purchased), unlike single-dimensional market-basket rules which relate values of one 'item' attribute. The constraint Age Group = 'Young' is a dimension-value (data) constraint: it slices the cube/table down to only the Young segment before any rule is generated.

### Step 2 — Constraint Analysis: Filtering to the Young Segment

| CID | Income Level | Policy Purchased  |
|-----|--------------|-------------------|
| C1  | High         | Health Insurance  |
| C3  | Low          | Travel Insurance  |
| C5  | Medium       | Vehicle Insurance |

After filtering, only 3 of the 5 records remain (C1, C3, C5), and each has a distinct Income Level paired with a distinct Policy — there is no repetition among the Young segment.

### Step 3 — Solution Development: Rules Within the Young Segment (base = 3 Young customers)

| Rule (within Age Group = Young)                                      | Support (of 3 Young) | Confidence |
|----------------------------------------------------------------------|----------------------|------------|
| Young $\wedge$ Income = High $\rightarrow$ Policy = Health Insurance | 1/3 = 33.3%          | 1/1 = 100% |
| Young $\wedge$ Income = Low $\rightarrow$ Policy = Travel Insurance  | 1/3 = 33.3%          | 1/1 = 100% |

| Rule (within Age Group = Young)                                                                                                                                                                                                                                                                                                                                                                                              | Support (of 3 Young) | Confidence |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|------------|
| Young $\wedge$ Income = Medium $\rightarrow$ Policy = Vehicle Insurance                                                                                                                                                                                                                                                                                                                                                      | 1/3 = 33.3%          | 1/1 = 100% |
| <p><b>Every Income–Policy combination within the Young segment is deterministic: confidence = 100% for all three rules, even though support of each individual rule is only 33.3% (since the Young segment itself is small, 3 of 5 customers).</b></p> <p><b>No single rule reaches a typical 50% support threshold, because the 3 Young customers split evenly across three distinct income levels with no repeats.</b></p> |                      |            |

## Step 4 — Justification / Application for Targeted Marketing

- High confidence despite low support is the key insight here: with more Young customer records, these perfect (100%) associations would very likely persist as strong, low-support-but-high-confidence 'niche' rules — exactly the kind of rule multidimensional mining is designed to surface, which a single flat min-support threshold (e.g. 50%) would otherwise discard as statistically insignificant.
- Business interpretation: Young + High income customers reliably buy Health Insurance; Young + Low income customers reliably buy Travel Insurance (possibly leisure/backpacking-related); Young + Medium income customers reliably buy Vehicle Insurance. A marketing team can use these as micro-segment targeting rules (e.g. bundle travel insurance offers with student/budget-travel promotions aimed at young, lower-income customers) rather than a single generic 'Young customers buy X' campaign.
- Recommended next step before deployment: because each rule currently rests on a single supporting record, the insurer should validate these patterns on a larger sample (raise the support base) before committing marketing spend — 100% confidence on  $n = 1$  is a promising signal, not yet statistical proof.